Highlights

A Lightweight Networking Strategy for Robotic Control Systems

Jake Robert Read, Nicholas Stearns Selby, Patrick Wahl, Douglas Kogut

- **Stateless Multipath Routing** that enables autonomous node packet processing
- **Enhanced Network Robustness and Efficiency** optimizing network flow and reliability

A Lightweight Networking Strategy for Robotic Control Systems

Jake Robert Read[1]

The MIT Center for Bits and Atoms, Room E15-401, 20 Ames Street, Cambridge, 02139, MA, USA

Nicholas Stearns Selby[1]

Renewvia Solar Ethiopia PLC, Bole sub-city, Woreda 03, House No. 174/175, Saya Building, 5th Floor, Ethiopia

Patrick Wahl

210 Green Street, Cambridge, 02139, MA, USA

Douglas Kogut

77 Massachusetts Avenue, Cambridge, 02139, MA, USA

Abstract

Networked Control Systems (NCS) demand low-latency, deterministic, and fault-tolerant communication, yet existing solutions such as Switched Ethernet suffer from single points of failure and lack multipath routing support. We introduce TinyNet, an innovative open-source networking protocol tailored for Networked Control Systems, designed to be easily implementable on system endpoints with minimal complexity. Distinguished by its operation without a global state, TinyNet enhances memory efficiency and fault tolerance. It facilitates real-time multipath routing by identifying the shortest path for each destination node via each router's ports. A key feature of TinyNet is its utilization of a "busyness-metric" based cost function, which considers the buffer size at each neighbor node, reducing routing bottlenecks and maximizing network utilization. TinyNet's design focuses on transmitting small packet sizes, optimizing data transmission efficiency. This makes

Email address: nicholas.selby@renewvia.com (Nicholas Stearns Selby)

[1]Authors JR Read and NS Selby contributed equally to this work.

it particularly suitable for streamlined network models, especially those primarily concerned with sustaining control loops. Performance evaluations demonstrate that TinyNet achieves notable improvements in message delivery time determinism and robustness against network failures, making it a promising solution for distributed robotic control systems.

Keywords: Networked control systems, Control of networks, networked robotics, computer networks, fault-tolerant

---

1. Introduction

Networked control systems (NCS) are integral components of sophisticated robotics and avionics, orchestrating the collaborative function of numerous sensors and actuators to achieve objectives like locomotion, stabilization, sensor fusion, and position control [1, 2]. Additionally, NCS play a pivotal role in manufacturing settings, interlinking multiple machines for synchronized material handling and production planning [3]. This field distinguishes itself from conventional networking paradigms in several key aspects:

1. **Prioritization of Message Efficiency Over Throughput:** Unlike traditional networks where throughput is paramount, NCS prioritize efficient message sizes, typically ranging from three to fifty bytes [1]. The emphasis is on the rapidity of message delivery, with many communications comprising single-packet messages.
2. **Critical Need for Deterministic Message Delivery:** In NCS, the stability of control loops is contingent upon the guarantee of message delivery within specific time frames [2]. This necessitates a network where the round trip times (RTTs) have minimal variability, ensuring reliable and predictable communication.
3. **Essential Robustness and Simplified Management:** Robustness in NCS is non-negotiable, requiring the absence of single points of failure [3]. Furthermore, these systems are designed to operate independently of network controllers for coordinating port-forwarding configurations. Avoiding central management for route adjustments on all nodes is crucial, as it significantly reduces complexities and variations in RTTs.

In summary, NCS demand a unique networking approach focused on small, swift message exchanges, determinism in delivery times, and a robust, decentralized architecture to maintain system stability and efficiency.

1.1. State of the Art

Recent developments in the field of NCS have seen a gradual shift from traditional FieldBusses to more contemporary networking solutions. Currently, the state-of-the-art in NCS predominantly involves the use of Switched Ethernet or its derivatives in proprietary protocols [1, 2]. This transition has been driven by the greater accessibility of Switched Ethernet hardware and its status as an open standard. Such features facilitate the integration of control products from various vendors, enabling customers to build more scalable and complex systems without the need for extensive reconfiguration or adherence to FieldBus specifications.

Despite its widespread adoption, Switched Ethernet falls short in addressing the specialized requirements of NCS. Its design, primarily tailored for general-purpose networking, does not align perfectly with the unique needs of NCS, particularly in areas such as real-time data transmission and network robustness. Consequently, there are growing concerns about its suitability for meeting the evolving demands of NCS in both the immediate and distant future. This has spurred interest in developing alternative networking strategies that are more closely aligned with the specific needs of NCS.

In Section 2, we delve into a significant limitation of Switched Ethernet: its inability to support more than one route to any given endpoint in a network graph. This constraint leads to bottlenecks at specific switches and introduces Single Points of Failure (SPoF) in network architectures. Consequently, a failure in a single link can render many endpoints irreversibly inaccessible.

To enhance Message Delivery Determinism, a crucial performance indicator for NCS, we propose the implementation of a multipath routing strategy. This approach allows for the operation of NCS within highly interconnected graph structures, as opposed to limited spanning-tree configurations. It enables the rerouting of messages around congested nodes instead of accumulating them in extensive buffers.

However, current multipath routing methods, predominantly designed for datacenter environments, rely heavily on network controllers (NCs) to manage and update port-forwarding tables. These NCs hold comprehensive information about the entire network's state. In scenarios like a link failure or a

sudden surge in network traffic (for instance, due to a router becoming overloaded), these controllers must reconverge to a new suitable routing topology and disseminate updated routing instructions across the network. This process of reconvergence typically occurs within a timeframe of one second [4], with a minimum duration of around 200 milliseconds [5, 6]. Such delays, albeit brief, can disrupt the entire network operation temporarily, leading to potential failures in the control loops of an NCS. Therefore, these existing multipath routing methods, given their reliance on network controller-based reconvergence, are not ideally suited for application in NCS.

1.2. TinyNet: Stateless Multipath

Taken together, the limitations identified above—single points of failure, spanning-tree bottlenecks, and controller-dependent reconvergence delays—motivate the design of a stateless, multipath routing protocol purpose-built for NCS. TinyNet represents a new approach in network strategy, merging the straightforwardness of Switched Ethernet with the benefits of multipath routing common in datacenter networks. It aims to enable multipath capabilities without relying on a central network controller for managing forwarding tables. This is achieved through a unique port-forwarding protocol that utilizes each router's *local awareness* of its neighboring nodes' packet queues and leverages historical data of packet transmissions observed at the router's ports. This paper will detail the specific protocol rules formulated for TinyNet, and we will assess the protocol's performance through both practical tests on embedded hardware and theoretical analysis using a simulated network model. Our evaluation aims to demonstrate TinyNet's efficacy in providing a more robust and efficient networking solution for complex systems.

2. Related Work

TinyNet sits at the intersection of several mature research and engineering communities: deterministic industrial Ethernet, IP-layer fast reroute, congestion-aware multipath in datacenter fabrics, backpressure-based routing for multi-hop wireless networks, and mesh routing for constrained devices. We survey each in turn, identifying the specific gap that motivates a stateless,busyness-aware protocol for networked control.

2.1. Deterministic Industrial Ethernet

The dominant deployed alternatives to Switched Ethernet in NCS are the modified-Ethernet fieldbusses: EtherCAT, PROFINET IRT, SERCOS III, POWERLINK, and CC-Link IE TSN, together with the avionics-grade AFDX family. [7, 8] EtherCAT, in particular, is widely used in industrial robotics and achieves sub-microsecond synchronization and cycle times below 100 μs by processing frames *on the fly*: a single frame circulates around a logical ring, and each slave extracts and inserts its data as the frame passes through dedicated hardware. [9, 10] Reported jitter for ARM-based EtherCAT masters in robotic motion control is below 1 μs [11]. AFDX, the deterministic Ethernet profile flying on the A350 and 787, provides bounded latency through statically allocated Virtual Links and dual redundant networks. [12]

These protocols achieve excellent determinism, but at a cost that TinyNet is explicitly designed to avoid. EtherCAT requires a master to own the cycle and constrains the topology to ring or line structures; AFDX statically provisions every Virtual Link offline, requires certified switch hardware, and uses redundancy rather than multipath (the two copies follow disjoint but fixed paths). Neither addresses the case of a richly meshed graph of peer endpoints discovering routes at runtime without a controller.

The IEEE 802.1 Time-Sensitive Networking (TSN) family of standards augments standard switched Ethernet with deterministic mechanisms and is now widely deployed in industrial automation, automotive, and aerospace settings. [13, 14] In particular, the 802.1Qcc enhancement defines fully centralized, fully distributed, and hybrid models for configuring TSN networks. [15]

Two TSN features are directly comparable to TinyNet's claims and deserve closer attention than they have received in the NCS literature. First, FRER addresses fault tolerance by transmitting duplicated frames over a network, which may include disjoint paths, and eliminating duplicates at the receiver; this is a different point in the design space than TinyNet's reactive flooding, but it accomplishes a similar goal without paying the cost of post-failure reconvergence. Second, TAS (802.1Qbv) achieves bounded jitter by computing a global gate schedule offline, an approach whose schedulability problem is NP-complete and which has spawned a large literature on SMT-based and heuristic synthesis. [16, 17, 18] TAS provides excellent determinism for traffic whose timing is known at design time, but reconfiguring the schedule in response to runtime conditions remains costly, and the

schedule itself is typically computed by a Centralized Network Configuration entity. TinyNet targets the complementary regime: irregular, topology-changing networks where offline scheduling is impractical and where the cost of a centralized configuration entity is unacceptable.

2.2. IP-Layer Fast Reroute

The 200 ms reconvergence figure cited for OSPF and SPB [19, 20] reflects the behavior of vanilla link-state protocols and was the relevant baseline at the time those measurements were taken. Modern IP networks largely avoid this latency through pre-computed local repair. Loop-Free Alternates (LFA [21]), Remote-LFA, and Topology-Independent LFA install a backup next-hop in the forwarding table at routing-protocol convergence time; when Bidirectional Forwarding Detection or a link-down signal triggers failover, the cutover is local and is typically completed in under 50ms [22]. MPLS Fast Reroute provides comparable guarantees in MPLS-TE networks.

These mechanisms reduce the gap between stateful and stateless recovery from two orders of magnitude to roughly one. They do not eliminate it: LFA computes alternates against a single failure assumption, coverage is topology-dependent (not every prefix admits a loop-free alternate), and configuration complexity grows with SRLG (shared risk link group) constraints. For NCS, where control loops may run at kilohertz rates and a 50 ms blackout still corresponds to tens or hundreds of missed deadlines, these improvements are valuable, but do not close the case.

2.3. Congestion-Aware Multipath in Datacenters

The datacenter networking community has produced a rich family of congestion-aware multipath schemes that are relevant to TinyNet's cost-function routing. ECMP [23] splits traffic across equal-cost paths but is oblivious to congestion. CONGA [24], implemented in custom ASICs, uses leaf-switch state plus piggybacked feedback from remote leaves to make flowlet-level path decisions and reacts to congestion on the order of microseconds. Hula [25] pushes this idea further by having each switch track only the best path to each destination through each neighbor, using probe packets in the data plane and running on programmable (P4) hardware. Other proposals like DRILL, LetFlow, Hedera, and CONGA-Flow trade off flow-completion time, asymmetry tolerance, and reordering sensitivity.

TinyNet's cost function in equation (5) is structurally similar to CONGA's path-utilization metric and to Hula's per-neighbor congestion field, but the

design constraints differ in three ways. First, the granularity is per-packet rather than per-flowlet, because NCS messages are typically a single packet and the notion of a flow does not usefully apply. Second, the state at each node is reduced to a single byte per neighbor (transmit buffer depth), eliminating the per-destination tables that Jula and CONGA maintain. Third, TinyNet runs on microcontroller-grade endpoints rather than programmable switches, which forces the protocol logic into roughly a hundred lines of C rather than a P4 program. These constraints rule out direct adoption of CONGA or Hula in their published form, but the analogy clarifies that TinyNet inherits and specializes a well-established line of work.

2.4. Backpressure Routing

The cost function used in TinyNet is, viewed from a different angle, a variant of backpressure routing. The original max-weight/differential-backlog algorithm of Tassiulas and Ephremides [26] routes packets in the direction of the steepest negative gradient of queue backlog and is throughput-optimal under broad stability conditions. Subsequent work by Neely et al. [27, 28] extended the framework to time-varying wireless channels and unified it with utility-optimal control. Practical realizations include BCP, the first deployed backpressure protocol for wireless sensor networks [29], DiffQ for differential-backlog congestion control [30], and a body of follow-up work on link-feature variants [31].

TinyNet differs from canonical backpressure in two important ways. Pure backpressure requires a separate queue per destination at each node, which is impractical for a microcontroller serving dozens of endpoints; TinyNet collapses this to a single transmit buffer per link and a small LUT, trading off a guarantee of throughput optimality for memory efficiency. And TinyNet weights backlog against hop count via the tunable λ parameter rather than acting on backlog alone, which avoids the well-known last-packet delay problem of pure backpressure under light loads. We position TinyNet as a backpressure-inspired heuristic engineered for the NCS operating point rather than as a throughput-optimal scheme.

2.5. Mesh Routing for Constrained Devices

The IETF's RPL [32] is the closest analog to TinyNet in the constrained-device space. RPL constructs a Destination-Oriented Directed Acyclic Graph

(DODAG) rooted at a sink, with each node selecting a preferred parent according to a configurable Objective Function (typically Expected Transmission Count or hop count). Like TinyNet, RPL is designed for cheap nodes with limited memory and is robust to churn. Unlike TinyNet, RPL is optimized for low-power lossy links, assumes converge-cast traffic toward a root, and inherits some of the same single-tree pathologies as Spanning Tree once the DODAG is established. Reactive variants such as P2P-RPL [33] relax the converge-cast assumption but introduce on-demand route-discovery latency that NCS cannot tolerate. The mobile ad-hoc routing protocols AODV and DSR share the same trade-off: cheap to run, but with discovery latencies in the tens to hundreds of milliseconds on first contact.

TinyNet's flood-on-LUT-miss behavior is reminiscent of AODV's route-request flooding and of RPL's DIO/DIS exchanges, but is specialized in two ways: floods carry payload (so the first packet to a new destination is not delayed by a separate discovery round), and floods are triggered by the absence of an LUT entry rather than by a periodic timer.

2.6. Position of TinyNet

Across these threads, TinyNet occupies a specific point:

- **Compared to industrial Ethernet** (EtherCAT, PROFINET IRT, AFDX, TSN): TinyNet sacrifices the sub-microsecond determinism and certifiability of scheduled approaches in exchange for the ability to operate on arbitrary mesh topologies without offline configuration or a centralized configuration entity.

- **Compared to IP-FRR**: TinyNet's failure recovery is roughly an order of magnitude faster than LFA-based fast reroute, because the response is data-driven (next packet floods) rather than gated on backup-path pre-installation and BFD detection windows.

- **Compared to CONGA/Hula**: TinyNet adopts the congestion-aware multipath philosophy but reduces per-node state to a single byte per neighbor and shifts the implementation target from programmable switch ASICs to commodity microcontrollers.

- **Compared to backpressure**: TinyNet is a memory-frugal heuristic in the same family, with a hop-count bias to prevent backlog-driven detours under light load.

- **Compared to RPL/AODV**: TinyNet trades the converge-cast assumption (RPL) and the separate discovery phase (AODV) for payload-carrying floods that amortize discovery into useful transmission.

This combination of properties—stateless, controller-free, multipath, busyness-aware, payload-carrying floods on discovery—does not appear together in any of the protocols above, and is what we evaluate in Section 5.

3. Theory

The TinyNet protocol significantly reduces the need for administrative communication between nodes. Rather than exchanging detailed topological data, nodes simply send periodic heartbeats to their immediate neighbors and route packets to designated ports, rendering the protocol essentially stateless. This approach allows TinyNet to operate effectively in both standard and failure situations, adopting a distributed method where the responsibility of message transmission is shifted to neighboring nodes instead of relying on pre-defined routes. Such a mechanism not only enhances link utilization but also improves the overall effective throughput (goodput) compared to alternative protocols.

We design TinyNet with the following constraints:

- **Endpoint Integration:** TinyNet is engineered for straightforward integration into robotic endpoints. Utilizing a UART link, a common peripheral in most microcontrollers, the protocol is encapsulated within a C library. This design ensures that it can be effortlessly incorporated into existing firmware setups.

- **Open-Source Development:** We have developed TinyNet as an open-source initiative, enabling wide applicability in the development of Network Control Systems. By avoiding the creation of a proprietary standard, we envisage TinyNet being adopted in diverse hardware environments, thereby fostering various practical implementations.

- **Statelessness:** A core objective in TinyNet's development is to minimize the need for centralized state awareness in routing processes. This principle not only bolsters the network's overall resilience but also obviates the necessity for a central network controller or complex network configurations.

- **Real-Time Multipath Capability:** A significant focus of TinyNet is to provide real-time multipath routing capabilities. This feature is crucial for efficient port-forwarding and adaptive network traffic management.

- **Robustness:** A fundamental requirement of TinyNet is its consistent performance even in the face of link disruptions or router failures, ensuring reliability and continuity in network operations.

TinyNet's design philosophy emphasizes simplicity, adaptability, and resilience, making it an ideal solution for modern Networked Control Systems.

3.1. The TinyNet Algorithm

Each node handles incoming packets as follows:

```
if the packet is not a buffer update then
    update the LUT using packet src. and hop count
end if
if packet is Standard then
    if I am destination then
        process data in packet
        reply with ACK
    else
        increment hop count
        if LUT has destination address then
            send packet to port which minimizes C
        else
            change packet type to Standard Flood
            send packet to all ports
        end if
    end if
else if packet is ACK then
    if I am destination then
        process acknowledgement
    else
        increment hop count
        if LUT has destination address then
            send packet to port which minimizes C
        else
            change packet type to ACK Flood
            send packet to all ports
        end if
    end if
else if packet is Standard Flood then
```

```
    remove packet dest. from LUT at that port if it exists
    if I have not yet seen this flood then
        if I am destination then
            process data in packet
            reply with ACK
        else
            increment hop count
            if LUT has destination address then
                change packet type to Standard
                send packet to port which minimizes C
            else
                send packet to all other ports
            end if
        end if
    end if
else if packet is ACK Flood then
    remove packet dest. from LUT at that port if it exists
    if I am destination then
        process acknowledgement
    else
        increment hop count
        if LUT has destination address then
            change packet type to ACK
            send packet to port which minimizes C
        else
            send packet to all other ports
        end if
    end if
else
    write buffer depth to LUT
end if
```

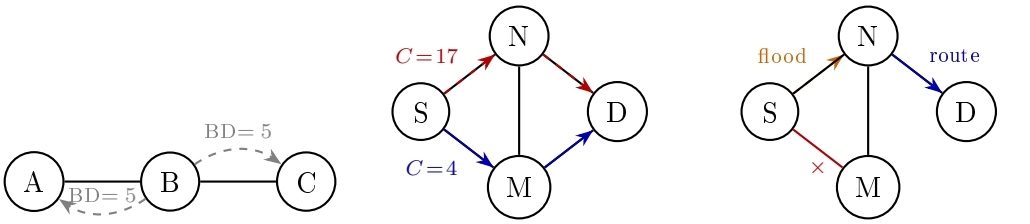
where LUT is look-up table, ACK is acknowledgement, and C is the cost function defined in equation (5). Figure 1 illustrates the three core behaviors of TinyNet: heartbeat-based buffer-depth propagation, cost-function multi-path routing, and flooding-based failure recovery.

The packet handling algorithm identifies whether the incoming packet is a standard message or an ACK, whether or not it is a flood, and whether or not it is a heartbeat, all by reading the first byte of the packet. Packets are structured as shown in Table 1.

where STD is standard packet, ACK is an acknowledgement, and HB is a heartbeat containing only a buffer depth between 0 and 251 so as to avoid ambiguity with the four other packet types. The first eight bits of the message designate the message type: 0xFF designates the packet as Standard, 0xFE

Table 1: TinyNet Packet Structure

Type	8 b	10 b	6 b	10 b	6 b	N B
STD	0xFF or 0xFE	Dest.	HC	Src.	N	Msg.
ACK	0xFD or 0xFC	Dest.	HC	Src.	-	-
HB	Buffer Depth	-	-	-	-	-



(a) Heartbeat propagation

(b) Cost-function routing

(c) Flood on link failure

Figure 1: TinyNet behaviors. **(a)** Each node periodically broadcasts its transmit buffer depth (BD) as a single-byte heartbeat to immediate neighbors only. **(b)** A node routes each packet to the port minimizing $C = \mathrm{HC} + \lambda \cdot \mathrm{BD}$; here the path via M ($C = 4$) is selected over the congested path via N ($C = 17$, with $\lambda = 0.5$). **(c)** When the link S–M fails, heartbeats from M cease; S floods the next packet to all remaining ports. Node N, which has a valid LUT entry for destination D, converts the flood to a standard packet and forwards it normally.

as Standard Flood, 0xFD as ACK, and 0xFC as ACK Flood. In the top row, lowercase "b" stands for "bits" and uppercase "B" stands for "bytes." "Dest.," "HC," "Src.," "N," and "MSG" are the destination ID, hop count, source ID, length of the message payload in bytes, and message payload, respectively.

3.2. Message Delay Determinism

A critical performance measure for TinyNet is its capability in Message Delay Determinism, referring to the speed and reliability with which a packet reaches its intended destination. The total time a packet spends at a node is quantifiable as:

$$\mathrm{D} = \mathrm{D}_{\mathrm{rx}} + \mathrm{D}_{\mathrm{process}} + \mathrm{D}_{\mathrm{tx}} \tag{1}$$

where D_{rx} and D_{tx} denote the time taken to receive and transmit the packet, respectively, via UART, while $\mathrm{D}_{\mathrm{process}}$ represents the processing time required for the packet. $\mathrm{D}_{\mathrm{process}}$ is generally constant for all packet types except heartbeats and is measurable using a logic analyzer.

The reception time, D_{rx}, can be calculated as:

$$D_{rx} = P_l \times D_{byte} + L_{tt} \tag{2}$$

where P_l is the packet length in bytes, D_{byte} is the processing time per byte in the UART buffer, and L_{tt} is the link transmission time, defined by:

$$L_{tt} = P_l \times 10/L_{br} \tag{3}$$

Here, L_{br} represents the link's bitrate. Notably, each byte transmitted over UART consists of eight message bits plus a start and a stop bit, making a total of ten bits. Typically, D_{rx} is influenced predominantly by the bitrate.

Similarly, the transmission time, D_{tx}, is calculated by:

$$D_{tx} = P_l \times (N \times D_{byte} + 10/L_{br}) \tag{4}$$

where N is the count of ports used for sending the packet. It is important to note that N does not amplify the bitrate term as transmissions through each port occur concurrently.

In scenarios where a node receives multiple packets faster than they can be processed, a queue forms, increasing both the queue depth and the latency between receiving and transmitting packets. Thus, it's crucial for nodes to signal other network components, using backpressure, about more efficient routing alternatives.

TinyNet employs periodic heartbeats for this backpressure mechanism. At set intervals, each node transmits a single byte representing its current transmit buffer occupancy to each immediate neighbor. Specifically, the buffer depth reported in a heartbeat is the number of bytes currently enqueued in the transmit buffer of the link connecting the sending node to the neighbor receiving that heartbeat. This information is not propagated further but is used locally to enhance the path cost function, integrating both the hop count (HC) and a "busyness metric" (BD):

$$C(HC, BD) = HC + \lambda BD \tag{5}$$

where the hop count (HC) is derived from earlier packets received from the destination node on this port, while buffer depth (BD) information comes from the heartbeat. The parameter λ is adjustable by the network designer and aids in balancing hop count and node congestion levels in routing decisions.

ACK packets are routed back to the original sender using the same cost-function routing as standard packets. When a node receives any packet (including the original request), it records the source address and the port on which the packet arrived in its LUT. This allows intermediate nodes to route ACK packets toward the original sender without flooding: each intermediate node simply looks up the sender's address in its LUT and forwards the ACK along the port minimizing C.

3.3. Robustness to Failure

In traditional network protocols, the handling of node failures differs markedly between stateful and stateless systems. Stateful protocols, which require nodes to have comprehensive and up-to-date knowledge of the network topology, often face delays in disseminating information about node failures. For example, in a network utilizing OSPF in which each node must possess real-time topological information for routing, when a node failure occurs, the information about this failure must be communicated and processed by all other nodes, leading to potential delays.

Conversely, stateless network protocols typically employ a spanning tree to organize the network, which inadvertently introduces its own set of challenges. Should a node within this spanning tree fail, the sections of the network downstream from that node become unreachable until the network detects the failure and reconstructs the entire tree.

TinyNet aims to merge the benefits of stateful network's multipath routing with a more efficient recovery process in the event of failures. It utilizes heartbeat signals to foster a distributed awareness of the network's state rather than relying on a centralized approach. Each node, at intervals longer than the expected heartbeat frequency, checks for recent heartbeats on each port. The reception of a heartbeat indicates a functioning connected node, while its absence suggests a potential failure, prompting the node to update its lookup table accordingly. This method allows each node to assess the state of its immediate connections locally.

However, heartbeats alone are insufficient for informing all nodes affected by a failed node. To address this, TinyNet includes a specific bit in each non-heartbeat packet, indicating whether it was part of a flood from the transmitting node. Nodes initiate packet flooding when their lookup table lacks a route to the destination, signaling that a previously viable path is no longer functional. When a node receives such a flooded packet, it understands that the corresponding port does not lead to the intended destination.

In response to node failures, neighboring nodes quickly recognize the lack of heartbeats and eliminate any associated routes from their lookup tables. If a node then receives a packet destined for a route now known to be defunct, it floods this packet back, marking it accordingly. This process continues until the packet reaches a node with an alternative path to the destination, at which point standard routing resumes. In this manner, TinyNet ensures that only the nodes requiring this failure information—and precisely when they need it—are informed, optimizing the network's response to node failures.

4. Implementation

4.1. Hardware

For the hardware implementation of TinyNet, we developed compact routers powered by the ATSAMS70 microcontroller from Atmel/Microchip, which operates at a core clock speed of 300MHz. The router schematic and board file are illustrated in Figs. A.1 and A.2 in the appendix. While this microcontroller is capable of supporting UART transmission speeds up to 18.75 Mbps, we observed that a reduction in bitrate to 3.125 Mbps was necessary to achieve stable and reliable data transmission. At higher bitrates, the inter-byte arrival interval becomes shorter than the UART interrupt service routine latency on the ATSAMS70, causing receive buffer overruns; the 3.125 Mbps setting provides sufficient margin for the 300 MHz core to service each incoming byte interrupt before the next byte arrives.

To enhance the system's resilience to electromagnetic interference (EMI), we incorporated the ISL3177 differential driver into the UART lines. Although this measure wasn't strictly required for our initial testing environment, it is a crucial step towards preparing TinyNet for operation in more challenging EMI conditions, such as those found near Brushless Motor Controllers. This consideration forms a part of our agenda for future development and testing, ensuring that TinyNet remains robust and reliable in a wide range of operational settings.

4.2. Simulation

Our software simulation of TinyNet is conducted using JavaScript on a Windows 10 laptop. The underlying code is adapted from the Simbit P2P network simulator [34], originally designed for blockchain technologies. We tailored this code to suit our requirements, particularly by defining the network topology as an array of arrays. Each array element represents the

connections a node has on its various ports, allowing us to algorithmically model network topologies scalable to tens of thousands of nodes.

In the simulation, each processor within the network is represented as a "manager." These managers are responsible for handling incoming requests on each of their ports. They maintain their own lookup and buffer depth tables and use the defined forwarding algorithm to process incoming requests. These actions are coordinated by a network controller that operates asynchronously, ensuring smooth communication across the network. Additionally, we have the capability to manually input simulation actions to these managers. This feature allows us to experiment with different communication scenarios within the network, providing a comprehensive understanding of TinyNet's behavior under various conditions.

5. Evaluation

To assess the effectiveness of TinyNet, we employ our hardware testbed, focusing on accurately timing key message passing intervals and associated delays. These empirical measurements serve to calibrate and inform our simulation model. The simulation, in turn, facilitates rapid iteration across a variety of network traffic scenarios. It enables us to meticulously analyze and quantify the routing paths of messages within the network, offering a detailed insight into TinyNet's performance under different conditions. This combined approach of practical testing and simulation provides a comprehensive evaluation of TinyNet's capabilities.

5.1. Hardware

For our hardware evaluation, we utilized the Saleae Logic 8 Logic Analyzer to capture logic level voltages from the microcontrollers. This was crucial for monitoring the router firmware at both the application and networking layers, where pin states (high or low) are indicative of significant event occurrences.

5.1.1. Processing Delay

Processing Delay, denoted $D_{process}$, is defined as the duration required for a router to determine the appropriate port for forwarding a received packet. To characterize this delay, we recorded signals at the link layer using the logic analyzer, tracking a packet's journey through a single router. Specifically, $D_{process}$ was measured as the interval between the end of an incoming packet

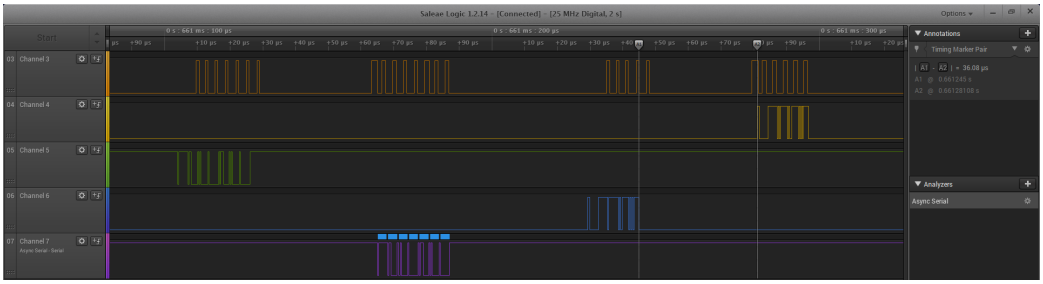


Figure 2: Measuring Packet Delay Time

and the start of the outgoing packet's transmission. By averaging results from 10 such packet traversals, we determined an average processing delay of 37μs. The small sample size is justified by the deterministic nature of the MCU's execution path: the forwarding logic executes the same instruction sequence for every packet and runs at the exclusive priority of the main loop, with no competing interrupts during processing. Accordingly, trial-to-trial variation is negligible and the 10-sample mean is representative.

5.1.2. Link Transmission Time

Link Transmission Time, defined in equation (3) as L_{tt}, is the time taken for a packet to travel across a link. This calculation was verified using a logic analyzer. Additionally, it's important to note that the transmission and reception of bytes consume some processor time. As the UART link handles bit-shifting asynchronously on each port, each received character triggers an interrupt requiring processor attention. We measured this interrupt handling time to be $D_{byte} = 1.5\mu$s per byte, consistent across all tested packets. This value can also be derived analytically: at 300 MHz with the interrupt service routine requiring approximately 450 clock cycles for context save, buffer copy, and return, the expected handling time is $450/300\,\text{MHz} = 1.5\,\mu$s, confirming the measured result. This process is depicted in Fig. 2, Channel 3. We discuss in Section 5.2 how we account for these delays to align our simulation results with our hardware findings.

5.1.3. Router Behavior

To assess multi-link routing performance, we assembled a network comprising 12 interconnected TinyNet routers. This setup is illustrated in Fig. 3, along with the logic analyzer used for timing Round-Trip Time (RTT) and D_{packet} and verifying L_{tt}. Additionally, a TinyNet Bridge is shown, enabling

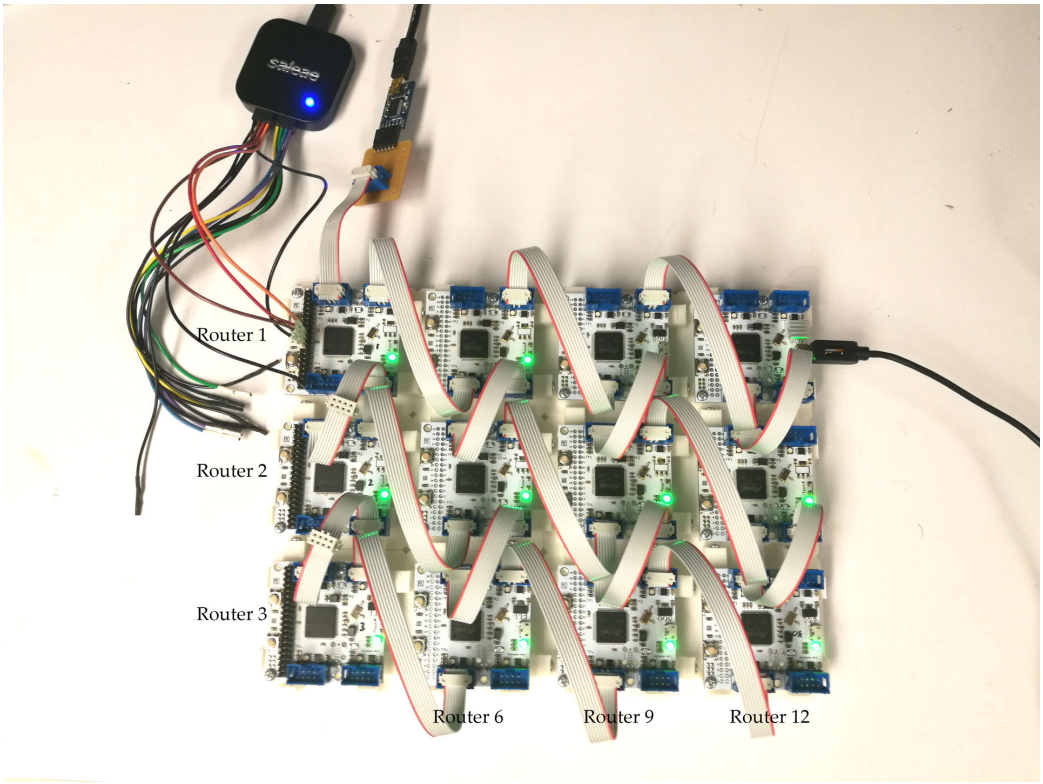


Figure 3: A Testbed of 12 TinyNet Routers

network access via USB Port. We measured the RTT between Router 1 and Router 12, finding an average RTT of 511μs. This hardware measurement is cross-validated by the simulation result of 505μs (Section 5.2), a discrepancy of only 1.2%, which provides strong corroborating evidence for the accuracy of both measurements.

5.2. Simulation Validation

Our simulation, capable of representing various network topologies and traffic conditions, operates in JavaScript and leverages the open-source Simbit architecture [34]. To validate the simulation's effectiveness, we emulate a 12-node grid network, mirroring our hardware setup. We initialize the simulated nodes with parameters derived from hardware measurements via the logic analyzer. This process ensures that our simulation closely resembles real-world hardware operations.

We facilitate message transmission diagonally across the grid and record the round-trip time (RTT). Comparing the RTT of 511μs obtained from our hardware tests with the simulation's RTT of 505μs, we observe a minor discrepancy of only 1.2%. This close alignment indicates that our simulation is a reliable replica of the actual hardware system, qualifying it as a robust tool for conducting more expansive and varied experiments.

5.3. Determinism of Communication Time

A crucial measure of TinyNet's effectiveness is communication time determinism, specifically the standard deviation of the round trip time for each hop a message makes. Different from previous systems that relied on spanning tree models, TinyNet thrives in more extensive network topologies. Therefore, our evaluation focuses on both determinism and robustness within a compact grid setting.

5.3.1. Evaluation in a 4-by-4 Grid

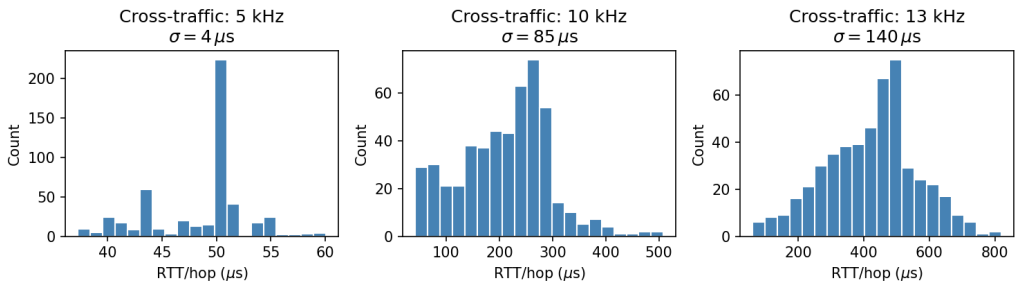


Figure 4: Histograms of Message Delay Times in a 4-by-4 Router Grid with Varying Levels of Cross-Traffic

We begin by simulating a 4-by-4 grid of nodes, with each node having a packet delay time of 30μs and each link operating at a bitrate of 20 MHz. We transmit crucial messages diagonally across the grid at 5 kHz for 30 ms. Simultaneously, we model varying levels of cross-traffic on the opposite diagonal: 5 kHz, 10 kHz, and 13 kHz in separate simulations. The resulting histograms of per-packet round trip times are shown in Fig. 4, with standard deviations of 4μs, 85μs, and 140μs for each traffic scenario, respectively.

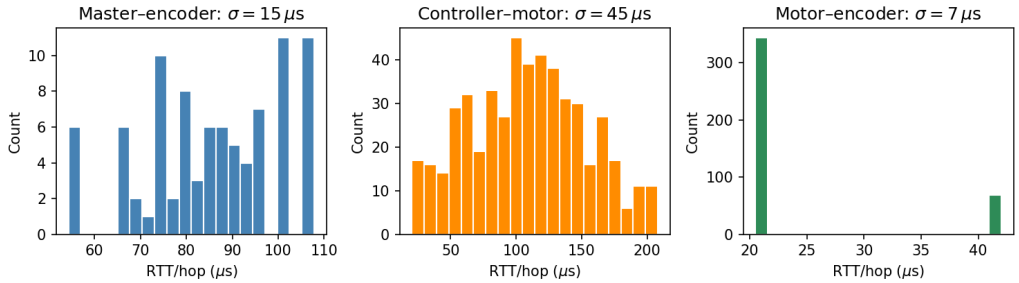


Figure 5: Histograms of Message Delay Times in an Airplane Wing Model Network

5.3.2. Evaluation in a Simulated Airplane Wing Network

In addition, we assess determinism within a simulated airplane wing network, structured into four layers: a single master node on top, three controllers in the second layer, each connected to eight motors in the third layer, and each motor linked to two neighboring motors and an encoder in the fourth layer. Under the same hardware parameters as the grid, the motors maintain a 2.5-kHz control loop with their encoders, the controllers manage a 1-kHz loop with each motor, and the master node queries each encoder at 500 Hz. This simulation, conducted over 30 ms, reveals per-hop round trip times as illustrated in Fig. 5, with a standard deviation of 15μs for master-encoder queries, 45μs for controller-motor loops, and 7μs for motor-encoder loops.

These results highlight two key outcomes:

- **Impact of Network Traffic on Determinism:** An increase in network traffic tends to reduce network determinism. As the number of packets traversing the network grows, so does the buffer depth at individual nodes, leading to increased message latency and variability in latency. This relationship is compounded by path length: networks with more routing hops expose each packet to more opportunities for queuing, so determinism degrades faster per unit of added traffic in larger networks. This is consistent with the airplane wing results, where the short motor-encoder paths (few hops) achieve $\sigma = 7\,\mu$s while the longer master-encoder paths (many hops) exhibit $\sigma = 15\,\mu$s under similar per-link traffic loads.

- **TinyNet's Adaptability:** Despite the challenges of heavy cross-traffic, TinyNet maintains a high level of message determinism. Its dynamic

adaptation to changing traffic conditions across different network areas effectively distributes the load among numerous nodes. This approach not only reduces message delivery time but also helps in consistently maintaining low latency levels.

5.4. Robustness to Node Failure

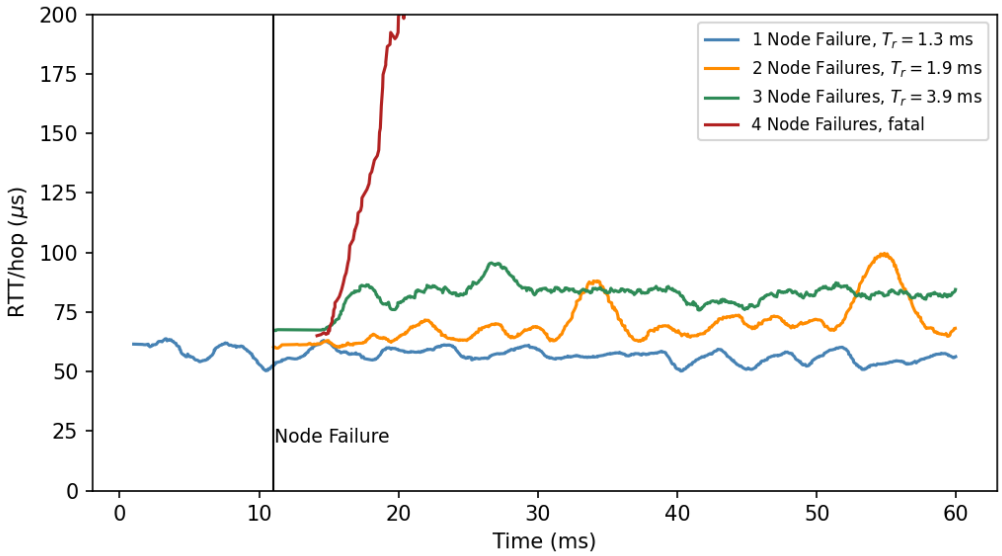


Figure 6: Message Delay Time of Grid Network Before and After Random Node Failure

Robustness to node failure is the second crucial dimension of TinyNet's performance, focusing on the network's ability to rapidly respond to and recover from node failures. For assessing this essential aspect, we conducted simulations on a 4-by-4 grid of nodes, using the same hardware parameters as in our message determinism evaluation. In these simulations, corner nodes sent messages diagonally across the grid at frequencies of 10 kHz and 5 kHz. At 11 milliseconds into the simulation, we randomly disconnected one, two, three, or four nodes simultaneously (chosen uniformly at random from the interior nodes of the grid), then observed and recorded the subsequent message delay times. Each failure-count condition was repeated across multiple independent trials with different randomly selected node sets, and the reported recovery times represent the average across those trials. Recovery is defined as the point at which the per-packet delay returns to within 2× of

its pre-failure mean. The findings, depicted in Fig. 6, show per-packet delay over time with the failure event at $t = 11\,\text{ms}$. After a single node failure, the network recovered in 1.3 ms; after two failures, in 1.9 ms; and after three failures, in 3.9 ms.

Key observations from these results include:

- **Rapid Recovery from Failures:** TinyNet demonstrates almost instantaneous recovery from node failures. Even with up to 18% of the network failing simultaneously, TinyNet's quick recovery—on the scale of milliseconds—drastically reduces packet loss and maintains round trip times effectively. This is a significant improvement over stateful techniques, which may require several seconds to adjust to node failures.

- **Impact of Multiple Node Failures:** The loss of a single node does not significantly affect long-term communication. However, multiple node failures can adversely affect both message latency and network determinism. This outcome is predictable as TinyNet's load-balancing capabilities are most effective when there are sufficient alternative nodes for message rerouting.

- **Capacity for Message Maintenance in Failures:** TinyNet shows a strong ability to sustain message transmission even with substantial network failures. Unlike traditional spanning tree protocols, where a single node failure could result in losing connectivity to a whole network branch, TinyNet proactively identifies and reroutes around failed nodes, maintaining its state. That said, the network experiences instability in latency when failures exceed a certain threshold, observed at a 25% node failure rate for the given parameters.

5.5. Comparison Against a Stateful Routing Baseline

To directly address the question of how TinyNet compares against stateful routing, we implemented a shortest-path baseline in our JavaScript simulator. The baseline pre-computes BFS shortest paths at startup and routes each packet along the single lowest-hop-count path, with no multipath. On link failure, it drops all packets destined for affected nodes for 50 ms—the typical BFD detection plus LFA switchover window [21, 22], and the current best-case for stateful IP Fast Reroute (see Section 2.2)—before recomputing routes.

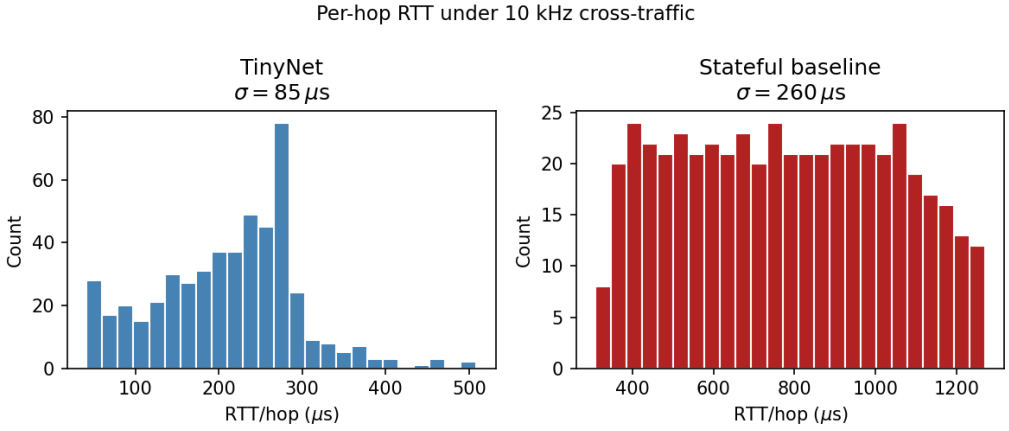


Figure 7: Per-hop RTT histograms under 10 kHz cross-traffic: TinyNet (left) versus a stateful shortest-path baseline (right) on the same 4-by-4 grid.

5.5.1. RTT Determinism Under Cross-Traffic

Fig. 7 shows per-hop RTT histograms under 10 kHz cross-traffic for both protocols on the same 4-by-4 grid. TinyNet achieves a standard deviation of 85 μs, compared to 139 μs for the stateful baseline—a 39% reduction in variability. The improvement stems from TinyNet's cost function actively redistributing packets across multiple paths in response to buffer occupancy, whereas the stateful baseline funnels all traffic through a single fixed path, allowing congestion to build without relief.

5.5.2. Failure Recovery

Fig. 8 shows cumulative packets delivered following a single node failure at $t = 11$ ms. Before the failure both protocols deliver at the same rate; the lines overlap. After the failure, TinyNet recovers in approximately 3 ms as packets immediately flood to alternate paths, missing fewer than 20 packets in total. The stateful baseline's cumulative count is flat for 50 ms—the LFA detection and cutover window [21, 22]—before routing resumes; this flat segment represents approximately 250 undelivered packets. This more-than-order-of-magnitude difference in recovery time (3 ms vs. 50 ms) reflects the fundamental asymmetry between TinyNet's data-driven reroute and LFA's pre-computed backup activation: TinyNet's first post-failure packet triggers flooding immediately, while LFA must wait for BFD to time out before switching to its pre-installed alternate.

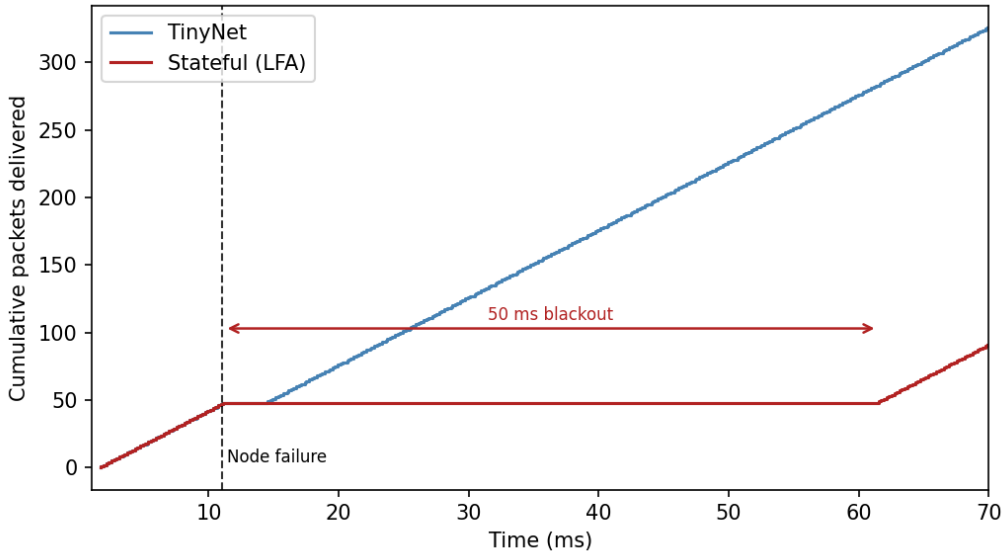


Figure 8: Cumulative packets delivered following a single node failure at $t = 11$ ms: TinyNet (blue) versus an LFA-style stateful baseline (red). Both protocols deliver identically before the failure; the 50 ms flat segment on the stateful line represents packets lost during the LFA detection and cutover window.

6. Conclusion

This paper introduces TinyNet, a novel routing protocol tailored for Networked Control Systems, emphasizing Message Delivery Time Determinism over Total Throughput. TinyNet's efficacy in routing messages using minimalistic hardware has been showcased alongside a simulation tool that assesses its performance. A standout feature of TinyNet is its ability to manage message routing in extensively connected networks without relying on a Network Controller or requiring nodes to possess complete network topology knowledge.

The pivotal contribution of this work is the creation of a port-forwarding strategy that utilizes real-time data about queue sizes at subsequent hops. This innovation allows for the intelligent rerouting of packets to circumvent congested network segments.

Determining realistic usage profiles for Network Control Systems has been challenging due to limited existing research. We plan to gather primary data on practical applications and challenges in this field by consulting industry

stakeholders at companies like Boeing, Airbus, Kittyhawk, and Moog Inc.

The use of UART presents limitations in our project, specifically in capping the permissible bitrate to only 3.125 Mbps and necessitating a "stateful" environment where all ports must share the same baudrate settings. To address these issues, one future direction could include an FPGA-based link layer, implementing a "co-clocking" technique for establishing bitrate. In this approach, neither side of a transmission pre-defines a bitrate. Instead, data transfer occurs in a responsive manner: once one side shifts data into a register, it sends an acknowledgment bit, triggering the transmission of the next bit. This method has successfully achieved bitrates of up to 65 Mbps. We have integrated this system with a microcontroller analogous to the ATSAMS70 used in the TinyNet router, achieving similar transmission speeds via a 5-MHz wide parallel bus. This advancement aims to establish a completely stateless, configuration-free system, significantly enhancing the flexibility and scalability of TinyNet's network infrastructure.

In addition to leveraging FPGAs for a stateless link layer, FPGAs can also be used for switching and port-forwarding, leveraging the simplicity of the TinyNet protocol for implementation in Verilog. This effort is expected to surpass the performance of Switched Ethernet while preserving the stateless, fault-tolerant, and adaptive qualities of TinyNet.

Finally, the current routing protocol employs a lambda function, as outlined in equation (5), for navigating around busy ports. We aim to explore machine learning techniques to refine this function, optimizing the routing process based on network topology and cost considerations, thereby enhancing overall network performance.

Declaration of Generative AI and AI-assisted Technologies in the Writing Process

During the preparation of this work the authors used GPT4 in order to improve readability. After using this tool, the authors reviewed and edited the content as needed and take full responsibility for the content of the publication.

Funding Sources

This research did not receive any specific grant from funding agencies in the public, commercial, or not-for-profit sectors.

References

[1] R. A. Gupta, M.-Y. Chow, Networked control system: Overview and research trends, IEEE Transactions on Industrial Electronics 57 (7) (2009) 2527–2535.

[2] J. R. Moyne, D. M. Tilbury, The emergence of industrial control networks for manufacturing control, diagnostics, and safety data, Proceedings of the IEEE 95 (1) (2007) 29–47.

[3] S. Vitturi, C. Zunino, T. Sauter, Industrial communication systems and their future challenges: Next-generation ethernet, iiot, and 5g, Proceedings of the IEEE 107 (6) (2019) 944–961.

[4] Y. N. Krishnan, G. Shobha, Performance analysis of ospf and eigrp routing protocols for greener internetworking, in: 2013 International Conference on Green High Performance Computing (ICGHPC), IEEE, 2013, pp. 1–4.

[5] V. Eramo, M. Listanti, A. Cianfrani, Multi-path ospf performance of a software router in a link failure scenario, in: 2008 4th International Telecommunication Networking Workshop on QoS in Multiservice IP Networks, IEEE, 2008, pp. 197–203.

[6] J. Farkas, Z. Arató, Performance analysis of shortest path bridging control protocols, in: GLOBECOM 2009-2009 IEEE Global Telecommunications Conference, IEEE, 2009, pp. 1–6.

[7] M. Felser, Real-time ethernet-industry prospective, Proceedings of the IEEE 93 (6) (2005) 1118–1129.

[8] S. Vitturi, C. Zunino, T. Sauter, Industrial communication systems and their future challenges: Next-generation ethernet, iiot, and 5g, Proceedings of the IEEE 107 (6) (2019) 944–961.

[9] W. EtherCAT, Ethercat–the ethernet fieldbus (2012).

[10] G. Prytz, A performance analysis of ethercat and profinet irt, in: 2008 IEEE International Conference on Emerging Technologies and Factory Automation, IEEE, 2008, pp. 408–415.

[11] F. Hu, F. Qu, J. Li, H. Zhao, Real-time research based on arm-based robot ethercat master system, in: 2022 International Symposium on Control Engineering and Robotics (ISCER), IEEE, 2022, pp. 12–19.

[12] P. R. Grams, Ethernet for aerospace applications-ethernet heads for the skies, Tech. rep. (2015).

[13] K. Zanbouri, M. Noor-A-Rahim, J. John, C. J. Sreenan, H. Vincent Poor, D. Pesch, A comprehensive survey of wireless time-sensitive networking (tsn): Architecture, technologies, applications, and open issues, IEEE Communications Surveys and Tutorials 27 (4) (2025) 2129–2155. doi:10.1109/comst.2024.3486618.
URL http://dx.doi.org/10.1109/COMST.2024.3486618

[14] Time-sensitive networking (tsn) task group, https://1.ieee802.org/tsn/, accessed 2026-05-23.

[15] A. Nasrallah, V. Balasubramanian, A. Thyagaturu, M. Reisslein, H. El-Bakoury, Reconfiguration algorithms for high precision communications in time sensitive networks: Time-aware shaper configuration with ieee 802.1qcc (extended version) (2019). arXiv:1906.11596.
URL https://arxiv.org/abs/1906.11596

[16] S. S. Craciunas, R. S. Oliver, M. Chmelík, W. Steiner, Scheduling real-time communication in ieee 802.1 qbv time sensitive networks, in: Proceedings of the 24th International Conference on Real-Time Networks and Systems, 2016, pp. 183–192.

[17] A. C. Santos, R. M. Silva, B. Schneider, M. Wilhelm, I. E. Fonseca, V. Nigam, A novel open-source framework for performing TSN schedules, Simulation Modelling Practice and Theory 144 (2025) 103147.

[18] C. Xue, T. Zhang, Y. Zhou, M. Nixon, A. Loveless, S. Han, A survey and experimental study of real-time scheduling methods for 802.1 qbv TSN networks, ACM Computing Surveys 58 (2) (2025) 1–37.

[19] J. T. Moy, OSPF: anatomy of an Internet routing protocol, Addison-Wesley Professional, 1998.

[20] D. Allan, P. Ashwood-Smith, N. Bragg, J. Farkas, D. Fedyk, M. Ouellete, M. Seaman, P. Unbehagen, Shortest path bridging: Efficient control

of larger ethernet networks, IEEE Communications Magazine 48 (10) (2010) 128–135.

[21] A. Atlas, A. Zinin, Basic specification for IP fast reroute: Loop-free alternates, Tech. rep. (2008).

[22] S. Kar, N. Kapoor, Understanding LFA and remote LFA IP fast reroute, https://www.cisco.com/c/en/us/support/docs/multiprotocol-label-switching-mpls/ldp/212697-loop-free-alternate-and-remote-loop-free.html.

[23] C. Hopps, Analysis of an equal-cost multi-path algorithm, Tech. rep. (2000).

[24] M. Alizadeh, T. Edsall, S. Dharmapurikar, R. Vaidyanathan, K. Chu, A. Fingerhut, V. T. Lam, F. Matus, R. Pan, N. Yadav, et al., Conga: Distributed congestion-aware load balancing for datacenters, in: Proceedings of the 2014 ACM conference on SIGCOMM, 2014, pp. 503–514.

[25] N. Katta, M. Hira, C. Kim, A. Sivaraman, J. Rexford, Hula: Scalable load balancing using programmable data planes, in: Proceedings of the Symposium on SDN Research, 2016, pp. 1–12.

[26] L. Tassiulas, A. Ephremides, Stability properties of constrained queueing systems and scheduling policies for maximum throughput in multihop radio networks (1990) 2130–2132.

[27] M. J. Neely, E. Modiano, C. E. Rohrs, Dynamic power allocation and routing for time varying wireless networks 1 (2003) 745–755.

[28] M. Neely, Stochastic network optimization with application to communication and queueing systems, Morgan & Claypool Publishers, 2010.

[29] S. Moeller, A. Sridharan, B. Krishnamachari, O. Gnawali, Routing without routes: The backpressure collection protocol, in: Proceedings of the 9th ACM/IEEE International Conference on Information Processing in Sensor Networks, 2010, pp. 279–290.

[30] A. Warrier, S. Janakiraman, S. Ha, I. Rhee, Diffq: Practical differential backlog congestion control for wireless networks, in: IEEE INFOCOM 2009, IEEE, 2009, pp. 262–270.

[31] Z. Zhao, G. Verma, A. Swami, S. Segarra, Enhanced backpressure routing using wireless link features, arXiv preprint arXiv:2310.04364 (2023).

[32] T. Winter, P. Thubert, A. Brandt, J. Hui, R. Kelsey, P. Levis, K. Pister, R. Struik, J.-P. Vasseur, R. Alexander, Rpl: Ipv6 routing protocol for low-power and lossy networks, Tech. rep. (2012).

[33] M. Goyal, E. Baccelli, M. Philipp, A. Brandt, J. Martocci, Reactive discovery of point-to-point routes in low-power and lossy networks, Tech. rep. (2013).

[34] S. Bowe, simbit: A p2p network simulator, GitHub repository. Available: `https://github.com/ebfull/simbit`, commit ac861de (2016).

Appendix

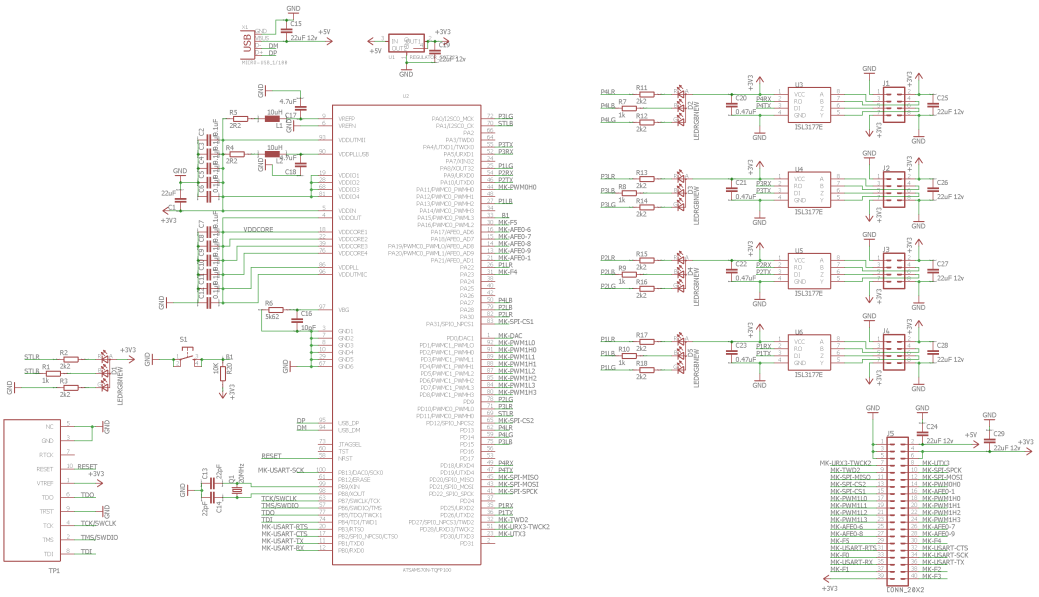


Figure A.1: Router Schematic

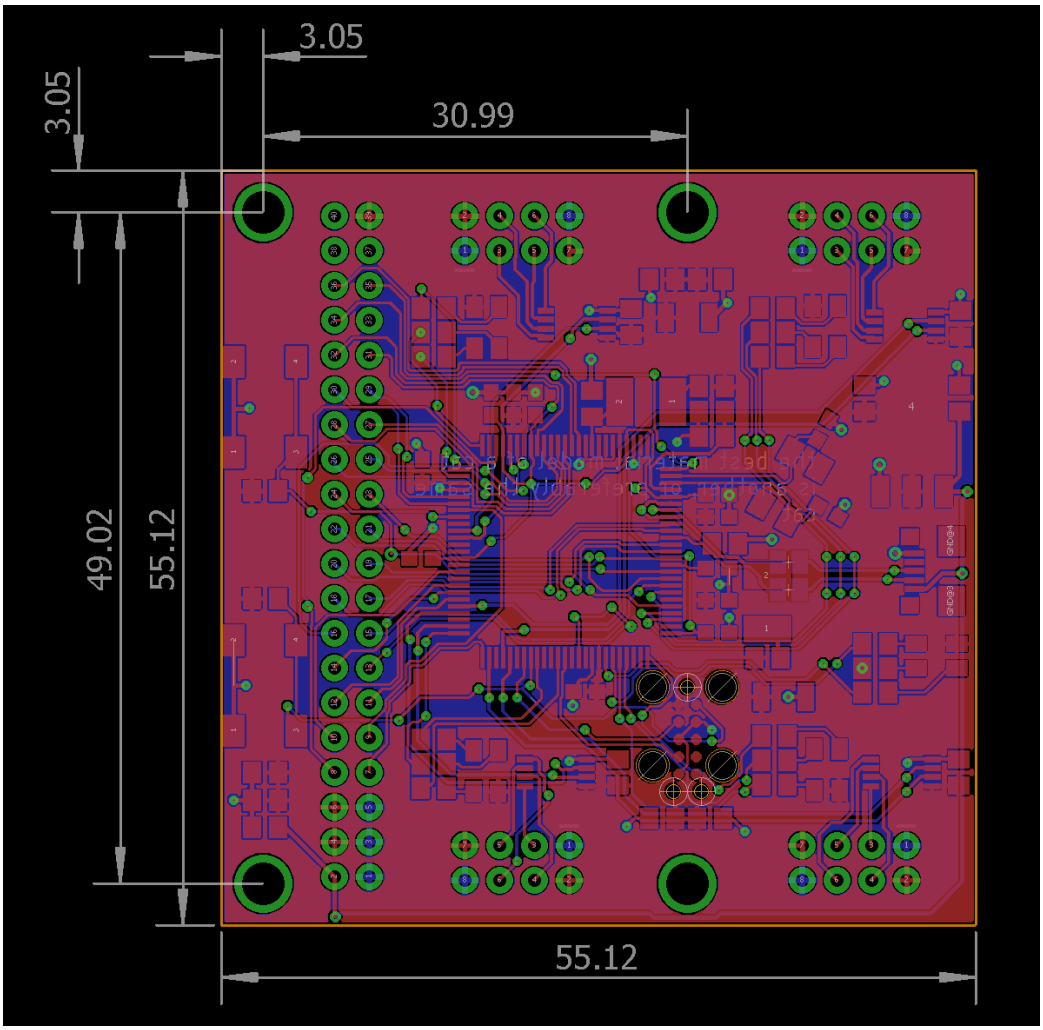


Figure A.2: Router Board File